

CSE 105

Data Structures & Algorithms I

Term Final Question Bank

Rahat Mohashin Zarif (2405115)

Tahsin Ahmad (2405061)

Table of Contents

Index	Topic	Question Count	Page
1	Asymptotic Analysis	11	4-6
2	Recurrence & Master Theorem	6	6
3	Singly Linked List	4	6-7
4	Doubly Linked List	4	7
5	Stack	10	7-9
6	Queue	4	9
7	Binary Tree	4	9-10
8	Binary Search Tree	11	10-11
9	Graph & Topological Sorting	6	11-12
10	Breadth First Search (BFS)	2	12-13
11	Depth First Search (DFS)	6	13-14
12	BFS or/and DFS	2	14
13	Heap	14	14-16
14	Priority Queue	3	16
15	Merge Sort	3	16
16	Quick Sort	6	17
17	More Sorting Algorithms	6	17-18
18	Divide & Conquer	3	18
19	Closest Pair of Points	3	18
20	Inversion & Significant Inversion	2	18-19

21	Multiplication & Squaring	3	19
22	Searching Algorithms	3	19
23	Dynamic Programming	3	19
24	Matrix Chain Multiplication	4	19-20
25	Longest Common Subsequence	4	20-21
26	Sequence Alignment	5	21
27	0-1 Knapsack Problem	3	22
28	Activity Selection Problem	4	23
29	Fractional Knapsack Problem	1	23
30	Minimum Spanning Tree	4	23-24
31	Greedy Algorithms	5	24-25
32	Disjoint Set & Union-Find	4	26
33	Miscellaneous	5	26-28

Note:

1. This Question Bank (QB) contains all the questions from the past seven term final examinations. As CSE 105 exam was held only for 3 years, the remaining 4 years questions are from CSE 203 course. Those questions are marked as CSE 203. The questions have been systematically organized by topic for ease of study and practice.
2. If a question involves multiple topics, it has been categorized either under the topic that appears earlier in our topic list or under the topic that is most dominant in the question. A small number of questions could not be clearly assigned to any single topic; these have been placed in the “Miscellaneous” section.
3. This Question Bank has been carefully sorted, edited, and formatted to ensure clarity and usability. However, if you notice any errors or inconsistencies, please feel free to inform us. Your feedback is highly appreciated.

DSA QB

Asymptotic Analysis

1. Analyze and compare the growth rate of $n^{\log \log n}$ and $\log^n n$. [22-23 4d] [Mark: 10]
2. Derive the time complexity of the following code fragments:

```
//code fragment 1
for (i=0; i<n; ++i) {
    ....for (j=0; j<i; ++j) {
        .....sum += i+j;
    ....}
}

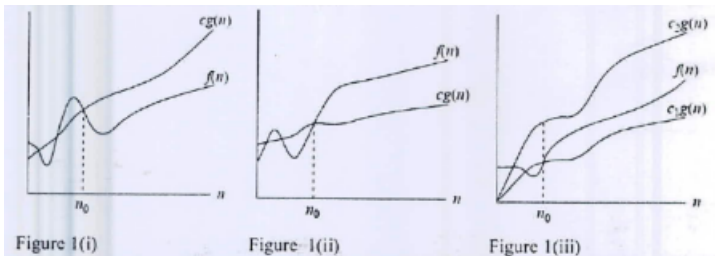
//code fragment 2
for (i=0; i<n; ++i) {
    ....for (j=0; j<i*i; ++j) {
        .....sum += i+j;
    ....}
}

//code fragment 3
for (i=0; i<n*n; ++i) {
    ....for (j=0; j<i; ++j) {
        .....sum += i+j;
    ....}
}

//code fragment 4
sum = 0;
for (k=1; k<=n; k*=2) {
    ....for (j=1; j<=k; j++) {
        .....sum++;
    ....}
}
```

[21-22 1a] [Mark: 16]

3. Explain what do the three figures, i.e., Figure 1(i-iii) signify with respect to asymptotic notations. [21-22 1c] [Mark: 5]



4. Find a tight upper bound on the complexity of the following code segments.

```
//segment i
void function (int n)
{
    ....int count = 0;
    ....for (int i=n/2; i<=n; i++)
        .....for (int j=1; j+n/2<=n; j=j++)
            .....for (int k=1; k<=n; k=k*2)
```

```

.....count++;
}
//segment ii
void function(int n)
{
....int count = 0;
....for (int i=n/2; i<=n; i++)
.....for (int j=1; j<=n; j=2*j)
.....for (int k=1; k<=n; k=k*2)
.....count++;
}

```

[21-22 5c] [CSE 203] [Mark: 10]

5. Deduce the time complexity of the following code fragments:

```

//code fragment 1
k=1;
while (n>1) {
....n=n/2
....k++;
}
//code fragment 2
sum=0;
for (k=1; k<=n; k*=2)
....for (j=1; j<=k; j++)
.....sum++;

```

[20-21 1a] [CSE 203] [Mark: 12]

6. Identify whether the following statements are true or false clearly justifying your answer:

- If an algorithm has time complexity $O(n^2)$ then it always makes n^2 steps, where n is the size of the input.
- An algorithm with time complexity $O(n)$ is always slower than an algorithm with time complexity $O(\log_2 n)$ for any input.
- An algorithm that makes $c_1 \log_2 n$ steps and an algorithm that makes $c_2 \log_4 n$ steps are in the same complexity class (c_1, c_2 are constants).
- An $O(2^n)$ algorithm can never be faster than an $O(n)$ algorithm. [20-21 1c] [CSE 203] [Mark: 12]

7. Express the following function in Θ notation with proper explanation:

$$f(n) = 5n - 2n^2$$

[19-20 4c] [CSE 203] [Mark: 7]

8. What does the term Asymptotic mean in complexity analysis? Can we show the worst case and average case time complexity using big-Theta notation? If yes, how and when can we do that? [19-20 7a] [CSE 203] [Mark: 15]

9. Consider the following code snippet for a binary search, where you are searching for an item x from an array a of size n . If all primitive operations (addition/subtraction, comparison, assignment, etc.) take one unit of time each to execute, what will be the time complexity of the above algorithm by counting all the primitive operations performed. Can you show the worst case time complexity of the above algorithm using big-Oh notation? [19-20 7b] [CSE 203] [Mark: 10]

```

BinarySearch(a, x)
1. left = 0
2. right = a.size()
3. while left < right
4. ....middle = (left + right) / 2
5. ....if a[middle] <= x then
6. ....left = middle + 1
7. ....else
8. ....right = middle

```

```

9. if left > 0 and a[left - 1] == x then
10. ....return true
11. else
12. ....return false

```

10. Derive the time complexity of the following code snippet, and write down the asymptotic complexity using big-Oh notation. Can we show the complexity using big-Theta notation? What is the difference in meaning between complexities while you use big-Oh or big-Theta? [19-20 7c] [CSE 203] [Mark: 10]

```

int i, j, k = 0;
for (i = n/2; i <= n; i++) {
    ....for (j = 2; j <= n; j = j * 2) {
        .....k = k + n/2;
    ....}
}

```

11. Suppose that you observe that a program takes 30 seconds to complete on inputs of size 600, 4.5 minutes to complete on inputs of size 1800 and 20 minutes to complete on inputs of size 2000. Develop a reasonable assumption for the order of growth of the running time. [18-19 5c] [CSE 203] [Mark: 10]

Recurrence & Master Theorem

12. Analyze the following recurrences using the Master Theorem,
- (i) $T(n) = 2T(\frac{n}{2}) + n \log n$
 - (ii) $T(n) = 2T(\frac{n}{2}) + n^2$ [23-24 5b] [Mark: 10]
13. Using the substitution method to solve the following recurrence:
 $T(n) = 4T(\frac{n}{2}) + 100n$
 [20-21 1b] [CSE 203] [Mark: 11]
14. Explain how a recursion tree can help substitution method of runtime analysis. [19-20 2b] [CSE 203] [Mark: 5]
15. Explain whether we can apply master method to any recurrence relation of the form $T(n) = aT(\frac{n}{b}) + f(n)$. If we cannot, explain a case where the theorem cannot be applied. [19-20 4b] [CSE 203] [Mark: 8]
16. Solve the following recurrences by using the Master theorem.
- $T(n) = 9T(\frac{n}{3}) + 5n\sqrt{n} + 8 \log n$
 - $T(n) = 3T(\frac{n}{4}) + 3n^2 + 4n + 5$
 - $T(n) = 4T(\frac{n}{2}) + 5n^2 + 7\sqrt{n}$
- [18-19 2a] [CSE 203] [Mark: 10]
17. Describe the worst case running time of the following code in "big-Oh" notation in terms of the variable n. You should give the tightest bound possible. [18-19 5b] [CSE 203] [Mark: 5]

```

int f(int n)
{
    ....if (n < 10)
        .....System.out.println("!"); return n+3;
    ....else
        .....return f(n-1) + f(n-1);
}

```

Singly Linked List

18. You are asked to merge two singly linked lists whose elements are sorted in ascending order. The output list will also be sorted in ascending order. Sketch an algorithm to solve the problem. Analyze that your algorithm run in linear time on the number of elements of the output list. [23-24 1a] [Mark: 18]

19. How would you reverse a singly linked list (without duplicating the list)? Write the steps (pseudocode) involved conceptually. [\[22-23 4a\]](#) [\[Mark: 10 \]](#)
20. How can you remove duplicate values from a sorted singly linked list? [\[22-23 4b\]](#) [\[Mark: 5 \]](#)
21. How can you check if a singly linked list is a palindrome? Write the steps involved conceptually. [\[22-23 4c\]](#) [\[Mark: 10 \]](#)

Doubly Linked List

22. You are given two linked lists of unsorted elements. Investigate whether you can develop a constant time algorithm to concatenate them if both of the lists are (i) singly linked lists (SLL), (ii) double linked list (DLL), (iii) circular SLL, (iv) circular DLL. If yes, sketch the algorithm and examine its correctness, otherwise demonstrate why a constant time algorithm is not possible. [\[23-24 1b\]](#) [\[Mark: 17 \]](#)
23. Write a function to delete a node with a given key from a doubly linked list. Assume the list contains unique keys and you have access to both the head and tail of the list. [\[22-23 3c\]](#) [\[Mark: 8 \]](#)
24. We use a doubly linked list (DLL) to keep records of some books. In the list, all the books that belong to the same publisher are placed in consecutive nodes. Now, write a function `addBook(title, publisher)` that inserts a book into the DLL as the first book for its publisher. So, if the list already has three books for a particular publisher, the 4th book will be inserted in front of the three books already in the DLL. For a new publisher, the book should be inserted at the end of the DLL. Assume that each DLL node has `prev` and `next` pointers and stores the `title` and `publisher` of a book as Strings. [\[20-21 4c\]](#) [\[CSE 203\]](#) [\[Mark: 10 \]](#)
25. Describe a situation where storing items in an array is clearly better than storing items on a linked list. [\[18-19 8d\]](#) [\[CSE 203\]](#) [\[Mark: 5 \]](#)

Stack

26. Assume that a string contains only left and right parentheses which may or may not be balanced and properly nested. For example, the string "`((()))()`" contains properly nested pairs of parentheses, but the string "`)()`" does not, and the string "`()`" does not contain properly matching parentheses. Using a stack, construct an algorithm that returns true if a string contains properly nested and balanced parentheses, and false otherwise. Modify the algorithm that returns the position in the string of the first offending parentheses if the string is not properly nested and balanced. That is, if an excess right parentheses is found, return its position; if there are too many left parentheses, return the position of the first excess left parentheses. Return - 1 if the string is properly balanced and nested. Assume indices start from 0. [\[23-24 4a\]](#) [\[Mark: 25 \]](#)
27. In an array based implementation of a stack when the array space runs out, the array capacity is increased. Now, analyze and explain the following scenarios through an amortized analysis of the `push()` function.
 - Increasing the Capacity of the Array by 1
 - Doubling the Capacity of the Array [\[21-22 1b\]](#) [\[Mark: 14 \]](#)
28. Suppose the numbers 0, 1, 2, ..., 9 were pushed onto a stack in that order, but that pops occurred at random points between the various pushes. Now you randomly wrote the following two sequences, S1 and S2 as example sequences in which the values in the stack could have been popped:
S1: 3, 2, 6, 5, 7, 4, 1, 0, 9, 8
S2: 3, 2, 6, 4, 7, 5, 1, 0, 9, 8
 Noticing this, your genius little brother winked at you and said that only one of your sequences is valid! Can you determine why? [\[21-22 3a\]](#) [\[Mark: 5 \]](#)
29. Consider the following C++ fragment. A stack is being used to parse the matching parentheses / brackets / braces- `()`, `[]`, and `{}`. Show the state of the stack at the end of this fragment.

```
1 #include <iostream>
2 using namespace std;
3
4 int main() {
5     ....const int M = 30;
```

```

6 ....int N = 1024;
7 ....int parent[N], height[N], depth[N];
8 ....int maxheights[100];
9
10 ....for (int i=0; i<100; ++i) {
11 .....maxheights[i] = 0;
12 ....}
13
14 ....double minmeandepth = 1e300;
15 ....double meandepths = 0;
16 ....double maxmeandepth = 0;
17
18 ....for (int j=0; j<M; ++j) {
19 .....for (int i=0; i<N; ++i) {
20 .....parent[i] = -1;
21 .....height[i] = 0;
22 .....}
23
24 .....for (int i=0; i<N-1; ++i) {
25 .....while (true) {
26 .....int p1 = rand() & (N-1);
27 .....int p2 = rand() & (N-1);
28
29 .....int s1 = p1;
30 .....int s2 = p2;
31
32 .....while (parent[s1
// yes, the code is actually as it is written in here :|

```

[21-22 3b] [Mark: 5]

30. Create a data structure `KStacks` that represents two stacks. Implementation of `KStacks` (`push1()` , `pop1()` , `push2()` , `pop2()`) should use only one array for storing elements efficiently. Also discuss the time complexity of the push and pop operations. **[21-22 5b] [CSE 203] [Mark: 10]**
31. Given a string consisting of opening and closing parentheses, find the length of the longest valid parenthesis substring. **[21-22 6b] [CSE 203] [Mark: 10]**
32. Write a program to reverse a stack using recursion, without using any loop. **[21-22 8c] [CSE 203] [Mark: 10]**
33. Suppose you are given an implementation of a list Abstract Data Type (ADT) which has the functions listed in Table A. Implement a stack which has the functionality shown in Table B. In your implementation, you can only use the list implementation of Table A. Assume that the elements are integers.

Table A: List

Function	Description
<code>insert(item)</code>	Inserts an element at the current location.
<code>append(item)</code>	Appends an element at the end of the list.
<code>remove()</code>	Remove and return the current element.
<code>moveToStart()</code>	Set the current position to the start of the list.
<code>moveToEnd()</code>	Set the current position to the end of the list.
<code>prev()</code>	Move the current position one step left. No change if already at the beginning.
<code>next()</code>	Move the current position one step right. No change if already at the end.
<code>length()</code>	Return the number of elements in the list.
<code>currPos()</code>	Return the position (in the list) of the current element.

Function	Description
<code>moveToPos(int pos)</code>	Set current position.
<code>getValue()</code>	Return the current element.

Table B: Stack

Function	Description
<code>clear()</code>	Reinitialize the stack, i.e., make it (logically) empty stack.
<code>push(item)</code>	Pushes an element.
<code>pop()</code>	Pop an element.
<code>length()</code>	Return the number of elements in the stack.
<code>max()</code>	Return the value of the maximum element in the stack.

[\[20-21 4a\]](#) [\[CSE 203\]](#) [\[Mark: 15 \]](#)

34. Your friend uses your above stack implementation regularly. A bug has been reported for the `max()` function of your stack implementation. Now your friend needs to implement a separate independent `max()` function which uses the stack data structure (avoiding its own `max()` function). He does not have access to the list data structure of Table A. How will he implement the `max()` function? He cannot use any data structure other than your stack implementation. [\[20-21 4b\]](#)

[\[CSE 203\]](#) [\[Mark: 10 \]](#)

35. Give the algorithm for converting an infix expression to postfix, evaluating a postfix expression using a stack. [\[18-19 6a\]](#) [\[CSE 203\]](#) [\[Mark: 10 \]](#)

Queue

36. Given an integer k and a queue of integers, how do you reverse the order of the first k elements of the queue, leaving the other elements in the same relative order? For example, if $k = 4$ and queue has the elements `[10, 20, 30, 40, 50, 60, 70, 80, 90]`; the output should be `[40, 30, 20, 10, 50, 60, 70, 80, 90]`. [\[23-24 4b\]](#) [\[Mark: 10 \]](#)

37. Implement a Circular Queue using an array in Java. Include the following methods:

- `enqueue(x)` - Adds an element x to the queue.
- `dequeue()` - Removes and returns the front element from the queue.
- `is_empty()` - Checks if the queue is empty.
- `is_full()` - Checks if the queue is full.
- `front()` - Returns the front element without removing it. [\[22-23 3b\]](#) [\[Mark: 15 \]](#)

38. Given a Queue data structure that supports standard operations like `enqueue()` and `dequeue()`, write a Stack data structure using only instances of Queue and Queue operations allowed on the instances. [\[21-22 6c\]](#) [\[CSE 203\]](#) [\[Mark: 10 \]](#)

39. Write a code snippet to implement a queue using a circular array, where each element of the queue contains the ID, name, and department of a student. [\[19-20 8a\]](#) [\[CSE 203\]](#) [\[Mark: 13 \]](#)

Binary Tree

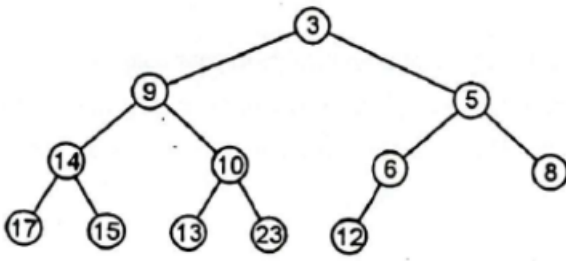
40. Analyze the maximum and minimum numbers of nodes in a binary tree of height h . [\[23-24 2b\]](#) [\[Mark: 10 \]](#)

41. Prove the following theorems that describe the properties of perfect binary trees:

- A perfect tree has $2^{h+1} - 1$ nodes
- The average depth of a node is $\Theta(\ln(n))$ [\[22-23 8a\]](#) [\[Mark: 10 \]](#)

42. Write a code snippet to traverse the binary tree in an order so that you can store it in an array. Show the placement of numbers of the following tree in an array. Write a code to access the parent of a given node. What will be the index no

of the array for node 12? [22-23 8b] [Mark: 25]



43. Given a Binary Tree, convert it to a Circular Doubly Linked List, where-

- The `left` and `right` pointers in nodes are to be used as `previous` and `next` pointers, respectively, in the converted Circular Linked List.
- The order of nodes in the List must be the same as in the order of the given Binary Tree.
- The first node of In-order traversal must be the head node of the Circular List. [21-22 5a] [CSE 203] [Mark: 15]

Binary Search Tree

44. Show the binary search tree (BST) that results from inserting the values 15, 20, 25, 18, 16, 5, and 7 (in that order). Write the enumerations of the tree that result from pre-order, in-order and post-order traversal of the tree. Write a recursive algorithm that takes as input the pointer to the root of a BST and a value `k`, and returns true if the value `k` appears in the tree and false otherwise. Modify the algorithm if the tree is a generalized binary tree. In both cases, analyze the time complexity of your algorithms. [23-24 2a] [Mark: 25]

45. You are given the following numbers to insert into an empty Binary Search Tree (BST):

5, 7, 8, 12, 15, 27

Determine which insertion order below would yield the tree with the least height?

- 15, 5, 27, 8, 7, 12
- 12, 7, 15, 27, 5, 8
- 8, 27, 7, 5, 15, 12
- 7, 5, 12, 8, 15, 27

[21-22 2a] [Mark: 5]

46. A perfect balanced BST requires all interior nodes to contain two children and all leaf nodes to be on the same level. A perfect balanced BST of $h=0$, would have $n=1$ node, $h=1$ would have $n=3$ nodes, $h=2$ would have $n=7$ nodes, and so on. Here, h is tree height and n is the number of nodes in the tree. Providing appropriate arguments, derive the worst-case running time, in terms of n , for successfully searching for an element in a perfect balanced BST. [21-22 2b] [Mark: 10]

47. Present a recursive method `countInRange(root, low, high)`, without any helper functions, to count, in a BST, all keys that are within a given range. Assume that keys in the BST are unique, i.e., there are no duplicates. For example, suppose that the keys in a BST are 4, 7, 10, 14, 15, 17, 20, 30. Then, `countInRange(root, 8, 17)` returns 4 and `countInRange(root, 21, 29)` returns 0. For your convenience a code snippet in C++ is given below, but, you need not follow any particular programming language.

```

public class BSTNode {
    ...int key;
    ...Value value;
    ...BSTNode left;
    ...BSTNode right;
}

// counts number of keys in BST that are in the range low to high, inclusive
// i.e. all keys k such that low <= k <= high
public static int countInRange(BSTNode root, int low, int high) {

```

```
....// COMPLETE THIS METHOD
}
```

[21-22 2c] [Mark: 10]

48. Present an algorithm to delete a full node in a binary search tree. Explain how it works with illustrative examples. [21-22 2d] [Mark: 10]
49. You are given two balanced binary search trees. Write a function that merges the two given balanced BSTs into a balanced binary search tree. Let there be m elements in the first tree and n elements in the other tree. Your merge function should take $O(m + n)$ time. [21-22 7b] [CSE 203] [Mark: 15]
50. While preparing for this exam, you have found an interesting question in a very old book with missing pages. It talks about a Binary Search Tree (BST) that stores integers in the range from 1 to 100 and proposes to search 22 and print the values encountered on the search path through the BST. Then it provides the following sequences of numbers (Sequences 1-3) and asks the following question: "Are these sequences possible search paths through the BST?"
- Sequence 1:** 100, 40, 20, 30, 23, 35, 22
- Sequence 2:** 100, 35, 15, 32, 20, 22
- Sequence 3:** 50, 60, 90, 40, 75
- You have to answer the above question. If your answer is 'yes', please show the path (with branches), if 'no', explain why. Hint: you were thinking whether there was a missing page with a figure showing the BST, but your genius little brother smiled and hinted that no need to check for a figure as the answer lied already within the sequences provided. [20-21 2a] [CSE 203] [Mark: 9]
51. In a Binary Search Tree (BST), successor of a node, N is defined as the node, N_{suc} with the smallest key greater than the key N . Similarly, the predecessor of N is defined as the node, N_{pre} with the largest key smaller than the key of N . Now answer the following questions.
- Write an efficient algorithm that will store the keys of the successor and predecessor of each node. Assume that you have two variables, `suc` and `pre`, respectively in the node class/data structure for that purpose.
 - Discuss an efficient implementation strategy of `removeFullNode(N, N_suc)` function (as defined below) with the help of appropriate illustrative examples. Analyze the time complexity of your strategy.
`removeFullNode (N, N_suc) : N` is a full node that has to be removed from the BST. N_{suc} is the successor of N .
 - Your genius little brother suggests that an equivalent strategy is available if we use N_{pre} instead of N_{suc} . Do you agree with him? Justify your answer using the same illustrative example you showed in the previous question. [20-21 2b] [CSE 203] [Mark: 19]
52. Suppose you have a set of numbers where you do a constant number of insertions and a linear number of lookups of the maximum over some time period. To maintain your set of numbers, you can choose between using either a heap or a binary search tree.
- Assume you are interested in minimizing the average total runtime over the time period. For each structure, what is the asymptotic upper bound on the average case runtime, and which of the two structures should you choose?
 - Suppose you realize that you are actually interested in the worst case. For each structure, what is the asymptotic upper bound on the worst case runtime, and which of the two structures should you choose now? [18-19 7b] [CSE 203] [Mark: 12] [This question will repeat in Heap]
53. Write a function `int treeHeight()` for a Binary Search Tree that computes the height of the tree. Give a worst-case big-Oh running time in terms of n , where n is the number of nodes in the tree. [18-19 8a] [CSE 203] [Mark: 8]
54. Write functions `void deleteMax()` for a Binary Search Tree that deletes the maximum element in the tree (or does nothing if the tree has no elements). Give a worst-case big-Oh running time in terms of n , where n is the number of nodes in the tree. Do not assume the tree is balanced. [18-19 8b] [CSE 203] [Mark: 7]

Graph & Topological Sorting

55. Given an adjacency list representation of a directed Graph, $G = (V, E)$ sketch an efficient algorithm to find its transpose graph $G^T = (V, E^T)$, where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. Analyze the time complexity of your algorithm. Also, evaluate how long you take to compute the in-degree and out-degree of every vertex of G . [22-23 1a] [Mark: 18]

56. Explain why topological sort algorithm cannot run on a directed graph that cycle(s). Derive the component graph of the directed graph G represented by the adjacency matrix given in Fig. for Q. No. 5(b) and 6. Then find the topological sorting of the vertices of the component graph. Show every steps. [21-22 5b] [Mark: 15]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Fig. for Q. No. 5(b) and 6

57. A DAG is given to you, you need to find the maximum number of edges that can be added to this DAG, after which new graph still remain a DAG that means the reformed graph should have maximum number of edges, adding even single edge will create a cycle in graph. [21-22 8b] [CSE 203] [Mark: 10]
58. Write a Graph (directed) data structure using a linked list. Write a delete node operation for the data structure. [19-20 5b] [CSE 203] [Mark: 15]
59. How do you determine whether a given graph is bipartite? [19-20 8b] [CSE 203] [Mark: 10]
60. Given a directed graph with N vertices and M edges that may contain cycles, write a function to find the lexicographically smallest topological ordering of the graph if it exists otherwise print -1 (if the graph has cycles). Lexicographically smallest topological ordering means that if two vertices in a graph do not have any incoming edge then the vertex with the smaller number should appear first in the ordering. [18-19 6d] [CSE 203] [Mark: 8]

Breadth First Search (BFS)

61. State the correctness theorem of Breadth-First Search (BFS). Use this theorem to judge whether the value $u.d$ assigned to a vertex u in BFS depends on the order of appearance of the vertices in the corresponding adjacency lists. Now demonstrate with the example of the graph given in Fig. for Q. No. 1(b) whether the breadth-first tree produced by BFS depends on the ordering of vertices in the adjacency lists if the same source vertex is used in every ordering. [22-23 1b] [Mark: 17]

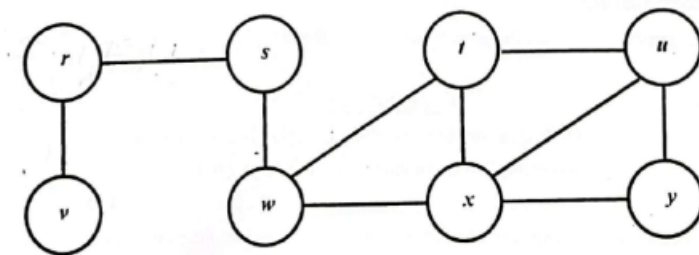


Fig. for Q. No. 1(b)

62. Consider the adjacency matrix representation of a directed graph G given in Fig. for Q. No. 5(b) and 6. Draw the graph and find its adjacency list representation considering the vertices in ascending order. Find the shortest path distance of every vertex from vertex 1 using BFS. Show the queue status when the vertex 10 is just enqueued. Prove that the distance of the most recent vertex (tail in the queue) is at most one larger than the distance of the oldest

vertex (head in the queue). [21-22 6a] [Mark: 20]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Fig. for Q. No. 5(b) and 6

Depth First Search (DFS)

63. Using a stack, construct an iterative (non-recursive) algorithm of depth first search (DFS) of a graph. Show how the state of the stack changes over time using the graph given in Fig. for Q. No. 3(b) assuming that the graph is represented using an adjacency list and your algorithm considers the vertices in alphabetical order. Analyze the complexity of your algorithm. Modify the algorithm so that it prints the type of each edge it encounters. [23-24 3b] [Mark: 20]

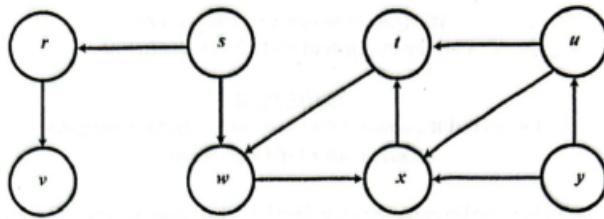


Fig. for Q. No. 3(b)

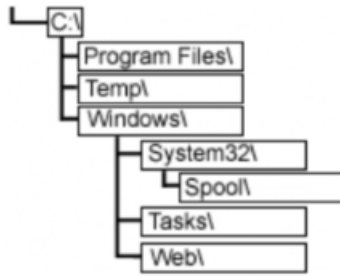
64. Define the four types of edges that a depth-first search (DFS) on a directed graph yields. Modify the pseudocode of the original DFS algorithm to print all the edges of a directed graph along with their types. Justify whether your algorithm will work correctly. [22-23 2a] [Mark: 18]
65. Classify the edges of the graph G represented by the adjacency matrix given in Fig. for Q. No. 5(b) and 6 after you run DFS on the graph considering the vertices in ascending order. Prove that an edge (u, v) is a tree edge if $u.d < v.d < v.f < u.f$. Notations bear their usual meaning. [21-22 6b] [Mark: 15]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Fig. for Q. No. 5(b) and 6

66. Consider the following hierarchical structure of a file system. Write a program to print the directory hierarchy of the file system, where all children of a parent will be placed one tab-space right from the parent. Use depth-first tree traversal

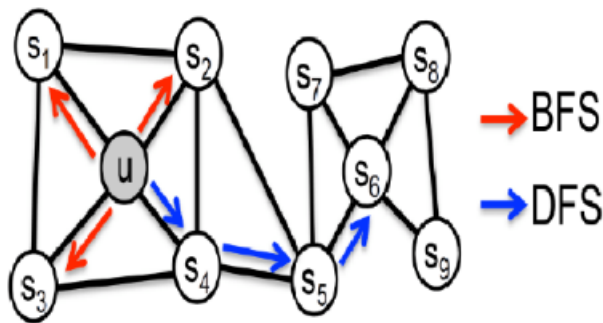
technique to perform the above task. [19-20 8c] [CSE 203] [Mark: 12]



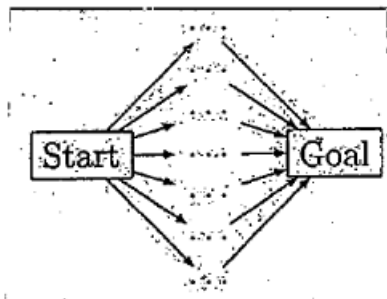
67. Explain when there is no back edge, if DFS is performed on an undirected graph. [18-19 7c] [CSE 203] [Mark: 8]
68. Give the running times of DFS in terms of $|V|$ and $|E|$, when using adjacency lists and adjacency matrix to represent the graph, respectively. [18-19 7d] [CSE 203] [Mark: 8]

BFS or/and DFS

69. You are given the following undirected graph that you need to traverse by following both BFS and DFS techniques. You will start the search from vertex u . Show the simulation steps of your algorithm for labeling the vertices in order of the visits. In case of BFS, the label of a vertex will be the level of the BFS search; whereas in case of DFS, you have to put two labels: first time and the last time of visiting the vertex. Also, write the pseudo-codes of BFS and DFS that you will be using to run the simulations. [19-20 5a] [CSE 203] [Mark: 20]



70. Suppose you have a graph like the one below. The dots signify some part of the graph that you don't know exactly, not a straight link. You know however, that there are many paths from the start to the goal. You also know that they tend to be rather long. Suppose that you want to implement a program that searches for a path and returns the first one it can find. You have no need for finding the optimal path in any sense, just any path will do. Would you want to use BFS or DFS as the basis for your program? Justify your answer. [18-19 8c] [CSE 203] [Mark: 10]



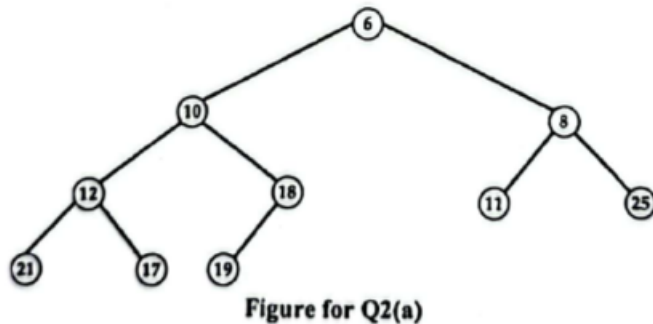
Heap

71. Assume you are given a set of integer numbers. Design a data structure and associated algorithms that support finding minimum and maximum of the numbers in constant time and insertion, deletion of maximum, and deletion of minimum in $O(\log n)$ time. Hints: use two different heaps on the same set of numbers; however remember their mutual dependencies. [23-24 3a] [Mark: 15]

72. Consider the following array (A) of integers. Verify whether the array is a max heap. Derive the complexity of your verification algorithm. Justify whether the algorithm `max_heapify (A, i)` will establish the max heap property of the array, if you apply algorithm with index `i=2`. Analyze the time complexity of `max_heapify` algorithm. [21-22 5a] [Mark: 20]

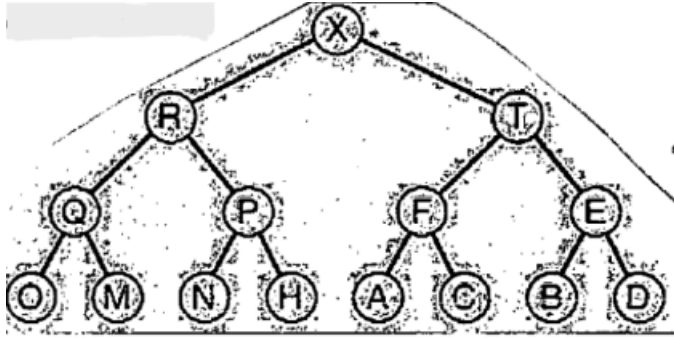
	1	2	3	4	5	6	7	8	9	10
A:	16	4	10	7	14	9	3	2	8	1

73. Calculate the number of swap operations (data interchanges) required if you apply `Build_Max_Heap` algorithm on the array
 $A = \{5, 3, 17, 10, 84, 19, 6, 22, 9\}$.
 Justify whether the max heap you produced can be used as a priority queue. Show that `Heap_Extract_Max` algorithm is essentially a part of Heapsort algorithm. Derive and compare the time complexities of these two algorithms. (`Heapsort` and `Heap_Extract_Max`). [21-22 8b] [Mark: 17]
74. Can you design a binary heap data structure (min-heap or max-heap) that supports finding both minimum and maximum elements in $O(1)$ time? Justify your answer. [21-22 1c] [CSE 203] [Mark: 7]
75. Calculate the min heap shown in Figure for Q2(a). Show the resulting min heap after deleting 10 from this heap. Show the intermediate steps. [21-22 2a] [CSE 203] [Mark: 10]



76. Given an array of integers, write a program to find the K-th largest sum of contiguous subarray within the array of numbers that has both negative and positive numbers. [21-22 7a] [CSE 203] [Mark: 10]
77. Assume that a binary heap is stored in an array called `treeNodes`, which has a capacity of 100 array elements, with array index 0 not being used. If the binary heap currently contains 86 keys, answer the following questions with proper explanation.
- Is `treeNodes[44]` a leaf node?
 - How many children does `treeNodes[43]` have?
 - Is the subtree rooted at `treeNodes[8]` a full binary tree?
 - How many levels are there in the subtree rooted at `treeNodes[8]` including `treeNodes[8]`?
 - Including the root node (`treeNodes[1]`) how many levels are there in the binary heap?
 - How many leaf nodes are there?
- [20-21 3a] [CSE 203] [Mark: 11]
78. Present and analyze the `BUILD_MAX_HEAP(array A)` function. Prove its correctness with the help of an appropriate loop invariant. [20-21 3b] [CSE 203] [Mark: 12]
79. Show the steps of building a max-heap by inserting the following numbers one-by-one 45, 36, 27, 14, 86, 19, 69, and 10. Also, show the memory representation of the heap. [19-20 6a] [CSE 203] [Mark: 15]
80. Write down the steps of sorting the above numbers (in Question 6 (a)) using heap-sort. [19-20 6b] [CSE 203] [Mark: 10]
81. Show the analysis how the build max-heap can be done in big-Theta (n), where n is the input size. [19-20 6c] [CSE 203] [Mark: 10]

82. Consider the following max-heap of letters and draw the results of inserting S. [18-19 7a] [CSE 203] [Mark: 7]



83. Suppose you have a set of numbers where you do a constant number of insertions and a linear number of lookups of the maximum over some time period. To maintain your set of numbers, you can choose between using either a heap or a binary search tree.
- Assume you are interested in minimizing the average total runtime over the time period. For each structure, what is the asymptotic upper bound on the average case runtime, and which of the two structures should you choose?
 - Suppose you realize that you are actually interested in the worst case. For each structure, what is the asymptotic upper bound on the worst case runtime, and which of the two structures should you choose now? [18-19 7b] [CSE 203] [Mark: 12] [This question is also under BST]
84. Write a method `isHeap()` which determines whether a particular binary tree is a heap. What is the big-Oh bound on the running time of this algorithm? [18-19 8e] [CSE 203] [Mark: 5]

Priority Queue

85. Given two max-priority queues represented by two arrays, write an efficient algorithm to merge them into a single max-priority queue. Analyze the time complexity of your algorithm. [22-23 2b] [Mark: 17]
86. The key at index 6 of the following max priority queue has been increased to 32. Re-establish the max-heap property of the queue and illustrate the steps using array only. Justify whether you can use max-heapifying algorithm here to re-establish heap property. [22-23 3a] [Mark: 12]

1	2	3	4	5	6	7	8	9	10	11	12	13
25	23	20	15	18	16	12	14	7	8	11	4	14

87. Assume that you are given a Max Priority Queue of n elements. Write a function that first finds and then deletes the minimum element of the priority queue. [18-19 4b] [CSE 203] [Mark: 10]

Merge Sort

88. Using the recurrence tree method, analyze the runtime complexity of the Mergesort algorithm to sort an array of size n . [23-24 6b] [Mark: 10]
89. Explain the run-time complexity of the merge sort algorithm. In your explanation, first, formulate the run-time of the merge-sort algorithm as a recurrence relation. Then solve the recurrence relation using both
- Recurrence tree method, and
 - Master theorem (distinguish the case that is applicable in your analysis). [22-23 6a] [Mark: 10]
90. Propose the pseudocode of merge sort algorithm where the array is divided in three equal parts at each step of divide. Derive the asymptotic runtime of this algorithm. Compare the runtime of this algorithm with the usual merge sort algorithm (i.e. where the array is divided in two equal parts at each step of divide) and mention the cases when one will outperform another. [19-20 2a] [CSE 203] [Mark: 15]

Quick Sort

91. Analyze the worst-case runtime of QuickSort algorithm under the following assumptions.
- Balanced partitioning
 - Imbalanced partitioning.
- In your analysis, formulate the worst-case runtime as a recurrence relation and solve the recurrent relations. [23-24 6a] [Mark: 10]
92. You are asked to sort the array of integers given below in ascending order. Use the Quicksort algorithm to sort the array. Highlight the pivot item at each step and show the status of the array after processing each pivot element.
[5, 17, 2, 4, 16, 8, 10, 6, 4, 26, 20, 7]
[22-23 6b] [Mark: 15]
93. QUICK-SORT is a divide-and-conquer based algorithm. The traditional PARTITION function of QUICK-SORT works by selecting a pivot element from the input array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the pivot. For this reason, it is sometimes called partition-exchange sort. The sub-arrays are then sorted recursively. This can be done in-place, requiring small additional amounts of memory to perform the sorting. Now answer the following questions.
- What is the worst case running time of QUICK-SORT algorithm if it is designed in a way that the last element of the input array is always selected as the pivot. Also give an example input scenario which will bring about the worst case running time.
 - Suppose, you have been asked to evaluate the performance of QUICK-SORT algorithm. You are using a bad input array which results in a very unbalanced partition at each of the recursive steps. Consider that at each recursive call, the PARTITION function is producing an α -to- β proportional split. Now using a recursive tree, find the time complexity of the QUICK-SORT algorithm for this scenario. If the last three digits of your student ID represents a number x , then consider the following.

$$\alpha = x \text{ if } x \leq 100$$

$$\alpha = 200 - x \text{ if } x > 100$$

$$\beta = 100 - \alpha$$
 Your answer must contain a well-drawn recursion tree where any kind of existing skewness of the tree is clearly illustrated. [20-21 6b] [CSE 203] [Mark: 12]
94. Apply partition procedure of Quicksort once on the following array, where the partition procedure chooses the element of the middle index (i.e. index $\lceil (p+1)/2 \rceil$) as pivot, assuming that the indices of the elements range from p to q (inclusive).
[10, 30, 50, 70, 90, 100, 80, 60, 40, 20]
You can assume that indexing of array starts from 0. Show all the steps.
Prove or disprove the fact that if we choose the element at middle index as pivot during partition (as stated above), the Quicksort algorithm will never have a runtime of $\Theta(n^2)$. [19-20 3a] [CSE 203] [Mark: 17]
95. Write the quick sort algorithm. Analyze the best case and worst case time-complexities of the algorithm. [18-19 3c] [CSE 203] [Mark: 15]
96. Give a concise accurate description of a good way for quicksort to choose a pivot element. Your approach should be better than "use the entry at location [0]". [18-19 6b] [CSE 203] [Mark: 5]

More Sorting Algorithms

97. Prove that any comparison-based sorting algorithm requires $O(n \log n)$ comparisons in the worst case. [23-24 6c] [Mark: 10]
98. Sort the following list of non-negative integers using Radix Sort (using LSD - Least Significant Digit first).
[329, 457, 657, 839, 436, 720, 355]
State the conditions required for Radix Sort to work correctly and complete in linear time. [23-24 7c] [Mark: 10]
99. Define stable sorting algorithm. Explain with an example. [23-24 8d] [Mark: 5]

100. Construct a proof to show that any comparison-based sorting algorithm requires $O(n \log n)$ comparisons in the worst case. [22-23 6c] [Mark: 12]
101. Describe a scenario where an "in-place" sorting algorithm is necessary and explain the reasons. [19-20 2c] [CSE 203] [Mark: 5]
102. Traditional insertion sort uses a linear search to scan (backward) through the sorted subarray $A[1 \dots j-1]$, where $A[j]$ is the element to be inserted next. Prove or disprove the idea that we can use a binary search instead to improve the overall worst-case running time of insertion sort to $\Theta(n \lg n)$. [19-20 4d] [CSE 203] [Mark: 5]

Divide & Conquer

103. Write an algorithm to find a peak element of a $n \times n$ matrix in $O(n)$ time using divide and conquer approach. A peak element of a 2-dimensional matrix is an element which has left, right, top and bottom elements (if available) lower than it. Define the recurrence relation and derive the runtime from it. [19-20 3b] [CSE 203] [Mark: 18]
104. Write a divide-and-conquer algorithm that finds the Maximum and the Minimum of an array of n distinct elements. First write the recurrence relation for the running time $T(n)$ of the algorithm, and then solve it by Recursion Tree Method. [18-19 1b] [CSE 203] [Mark: 10]
105. Consider a divide-and-conquer algorithm as follows: if the problem size is less than or equal to 5, then it solves the problem at constant time. Otherwise, it divides the problem of size n into 4 subproblems, each with a size of $n/3$. This division takes $O(\sqrt{n} \log n)$ time. The algorithm then solves the subproblems recursively, and merges their solutions at time $O(n)$. Write the recurrence relation for the running time $T(n)$ of this algorithm. [18-19 2a] [CSE 203] [Mark: 10] [Second half of the question is under Master Theorem]

Closest Pair of Points

106. Given n points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ in the plane, give an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm. [21-22 8a] [Mark: 18]
107. Given points $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$ in the plane, given an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm. [21-22 3a] [CSE 203] [Mark: 20]
108. Suppose, in a 2-dimensional co-ordinate system, you are given with n points, say $P_1, P_2, \dots, P_n$, where the x-y co-ordinates of any point P_i is represented as (x_i, y_i) . You have been assigned to find the closest pair of points using divide and conquer method.
- Write down the approach that you will follow to split these n points into two partitions, say LEFT and RIGHT, in such a way that each partition contains roughly $\frac{n}{2}$ points.
Now suppose, the LEFT and RIGHT partitions, respectively calculate d_L and d_R as the distances between the closest pair of points within themselves. Now your task is to check whether there exist a pair of points P_{Li} and P_{Rj} such that $P_{Li} \in \text{LEFT}$ and $P_{Rj} \in \text{RIGHT}$, and the distance between P_{Li} and P_{Rj} is less than $\min(d_L, d_R)$.
 - Write down the steps to perform this checking so that the minimum number of comparisons is ensured and the checking can be performed in $O(n)$ time. [20-21 8a] [CSE 203] [Mark: 15]

Inversion & Significant Inversion

109. Given a sequence of n numbers $a_1, \dots, a_n$, which we assume are all distinct, we define an inversion to be a pair $i < j$ such that $a_i > a_j$. Present an $O(n \log n)$ divide-and-conquer algorithm to count the number of inversions between two orderings. One might feel that this measure of inversion is too sensitive and may want to change it. Let's call a pair a modified-inversion if $i < j$ and $a_i > 2a_j$. What modifications do you need to make in the divide-and-conquer algorithm (for counting inversion) to count the number of modified-inversions? [21-22 7b] [Mark: 20]
110. In the problem of finding the number of inversions, we are given a sequence of n distinct numbers $a_1, a_2, \dots, a_n$. We define an inversion to be a pair $i < j$ such that $a_i > a_j$.
Let's call a pair a significant inversion if $i < j$ and $a_i > 2a_j$. Give an $O(n \log n)$ algorithm to count the number of

significant inversions of any given sequence of n numbers. You have to write the corresponding pseudo-code of your algorithm. [20-21 8b] [CSE 203] [Mark: 10]

Multiplication & Squaring

111. Using the Master Theorem, prove that Strassen's algorithm improves the runtime of multiplying two $n \times n$ matrices compared to the naïve approach with time complexity $\theta(n^3)$. [23-24 6d] [Mark: 5]
112. Provide a divide-and-conquer algorithm that takes an n -digit number A as input and efficiently computes the square of A with a faster asymptotic running time than $O(n^2)$. Analyze the running time of your algorithm. [21-22 1b] [CSE 203] [Mark: 12]
113. "It is asymptotically faster to square an n -bit integer than to multiply two n -bit integers." Do you agree with this statement? Justify your answer. [21-22 2c] [CSE 203] [Mark: 10]

Searching Algorithms

114. Why is Binary Search preferred over Ternary Search? [21-22 7c] [CSE 203] [Mark: 10]
115. You are given two sorted lists of size m and n . Give an $O(\log m + \log n)$ time algorithm for computing the k th smallest element in the union of two lists. [18-19 5a] [CSE 203] [Mark: 12]
116. Ternary search, like binary search, is a divide-and-conquer algorithm. It is mandatory for the array (in which you will search for an element) to be sorted before you begin the search. In this search, after each iteration it neglects $\frac{2}{3}$ part of the array and repeats the same operations on the remaining $\frac{1}{3}$. Write an algorithm for ternary search and show its complexity. [18-19 6c] [CSE 203] [Mark: 12]

Dynamic Programming

117. Assume that you have obtained a gold bar of integer length n . You plan to cut it into smaller pieces and sell them to the market. Further assume that, for any integer $i \leq n$ you know the profit you will obtain if you sell a gold bar of length i to the current market. You are now facing the problem of finding the maximum profit that you can obtain by cutting the bar into smaller pieces and selling them.
- Using the cut-and-paste method, formulate an argument to show that the problem of obtaining the maximum profit by cutting this gold bar into smaller pieces (and selling them) has optimal substructure property.
 - Construct a recursion tree diagram to show that the problem has overlapping subproblem property. [23-24 5a] [Mark: 15]
118. Discuss a problem scenario where adopting memorization over dynamic programming can result in a faster solution. Justify your answer. [20-21 8c] [CSE 203] [Mark: 10]
119. Briefly explain the steps that are to be followed for solving a problem using dynamic programming method. [18-19 3a] [CSE 203] [Mark: 10]

Matrix Chain Multiplication

120. You are given a sequence of matrices $\langle A_1, A_2, \dots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to determine the product of the given chain of matrices. Formulate a recurrence relation to solve this problem. Design an efficient dynamic programming algorithm using the recurrence relation. [23-24 8b] [Mark: 10]
121. You are given a sequence of matrices $\langle A_1, A_2, \dots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to determine the product of the given chain of matrices.
- Formulate arguments to show that the MCM problem has optimal substructure property.
 - Compose a recursion tree diagram to show that MCM has overlapping subproblem property.
 - Design an efficient algorithm using dynamic programming approach to solve this problem. [22-23 5a] [Mark: 25]

122. Matrix Chain Multiplication (MCM) is an optimization problem that finds the most efficient way to multiply a given sequence of matrices. The problem is not actually to perform the multiplications but merely to decide the sequence of the matrix multiplications involved. The matrix multiplication is associative as no matter how the product is parenthesized, the result obtained will remain the same. However, the order in which the product is parenthesized affects the number of simple arithmetic operations needed to compute the product. The popular MCM problem aims to find the optimal parenthesization so that the number of arithmetic operations is minimized. We have learned in the class that using dynamic programming, we can design an $O(n^3)$ algorithm to find the optimal parenthesization and the required number of arithmetic operations. Now answer the following questions.

- Suppose $m[i, j]$ represents the minimum number of multiplications needed to compute $A_i \cdot A_{i+1} \cdot \dots \cdot A_j$. If we are given n such matrices, then our goal is to compute $A_1 \cdot A_2 \cdot \dots \cdot A_n$ or we can say to find the value of $m[1, n]$. Now, derive a recursive definition to find $m[i, j]$, which we can utilize to populate our DP table.
- With the help of recursion tree, describe the overlapping sub-problem property of the MCM problem.
- Suppose you have to multiply the following six matrices with the help of dynamic programming (DP). The number of rows and columns of each of the matrices are provided within the adjacent parenthesis.

$A_1(30 \times 35)$

$A_2(35 \times 15)$

$A_3(15 \times 05)$

$A_4(05 \times 10)$

$A_5(10 \times 20)$

$A_6(20 \times 25)$

You are also given with the following partially filled DP table which aims to solve the MCM problem. The unpopulated cells are marked by $m[i, j]$, where i and j represents their row and column indices, respectively. First, give an ordering how you will populate these marked cells. Second, using the values of the already populated cells, calculate the value of $m[2, 5]$. [20-21 6a] [CSE 203] [Mark: 23]

	1	2	3	4	5	6
1	0	15750	7875	$m[1, 4]$	$m[1, 5]$	$m[1, 6]$
2		0	2625	4375	$m[2, 5]$	$m[2, 6]$
3			0	750	2500	$m[3, 6]$
4				0	1000	$m[4, 6]$
5					0	$m[5, 6]$
6						0

Table for Ques No. 6(a) – Partially filled DP table for MCM problem

123. Let $A_1(1 \times 4)$, $A_2(4 \times 5)$, $A_3(5 \times 3)$, $A_4(3 \times 6)$, $A_5(6 \times 7)$, $A_6(7 \times 1)$ be six matrices. Compute $m[1, 6]$ by assuming $m[1, 3] = 35$, $m[2, 4] = 132$, $m[1, 5] = 95$, $m[2, 5] = 270$, $m[2, 6] = 95$, $m[4, 5] = 126$, $m[4, 6] = 60$. [18-19 2c] [CSE 203] [Mark: 13]

Longest Common Subsequence

124. Given the two strings, $X = \text{"AGGTAB"}$ and $Y = \text{"GXTYAB"}$ construct the dynamic programming (DP) table to compute the length of the Longest Common Subsequence (LCS) between X and Y . Clearly state the LCS and its length. [23-24 8a] [Mark: 10]
125. Given two strings, the longest common subsequence problem is to find the longest subsequence present in both strings. A common subsequence is a sequence that appears in both string in the same relative order, but not necessarily contiguous. For example, for two strings $S_1 = \text{"ALGORITHM"}$ and $S_2 = \text{"LITHIUM"}$, the longest common subsequence is "LITHM". Present a dynamic programming algorithm to find the length of the longest common subsequence of two strings. Derive a proof of correctness of the algorithm and also derive its running time. [21-22 4b] [Mark: 17]
126. Given two strings, the longest common subsequence problem is to find the longest subsequence present in both strings. Note that unlike substrings, subsequences are not required to occupy consecutive positions within the original sequences. A common subsequence is a sequence that appears in both string in the same relative order, but not

necessarily contiguous. For example, for two strings $S_1 = \text{"ALGORITHM"}$ and $S_2 = \text{"LITHIUM"}$, the longest common subsequence is "LITHM". Give a dynamic programming algorithm to find the length of the longest common subsequence of two strings. [21-22 4b] [CSE 203] [Mark: 20]

127. Find all longest common subsequences of $X = \langle TAAGUATU \rangle$ and $Y = \langle AUAGTUT \rangle$ by using dynamic programming method. Also find a shortest common super-sequence of X and Y using an LCS of X and Y . The shortest common super-sequence problem is defined as: Given two sequences X and Y , find the shortest sequence that has both X and Y as subsequences. For example, for two sequences $X = \langle BUET \rangle$ and $Y = \langle CSE \rangle$ a shortest common super-sequence is $\langle BCUSET \rangle$. [18-19 4c] [CSE 203] [Mark: 15]

Sequence Alignment

128. Using a dynamic programming algorithm, determine the minimum cost alignment of the two strings $X = \text{ATCGGT}$ and $Y = \text{ACCDT}$ where both mismatch penalty and gap penalty are 1 and the cost of a match is 0. You do not need to write the algorithm, just show the dynamic programming (DP) table. Show the alignments and mark the paths (which correspond to the alignments), in the DP table. [22-23 5b] [Mark: 10]
129. Compute the best alignments (i.e., alignments with the minimum edit distance) of two DNA sequences, ATCGGT and ACCGT using dynamic programming. Use 2 as the gap penalty, 1 as the mismatch penalty, and 0 as the penalty for a match. If there are multiple optimal alignments, find all of them. You do not need to write the algorithm, just show the dynamic programming (DP) table. Show the alignments and mark the paths (which correspond to the alignments), in the DP table. [21-22 4a] [Mark: 18]
130. Find the best alignment (i.e., an alignment with the minimum edit distance) of the following two DNA sequences using dynamic programming. Use 3 as the gap penalty, 2 as the mismatch penalty, and 0 as the penalty for a match. You do not need to write the algorithm, just show the dynamic programming (DP) table. Show the alignment and mark the path, which corresponds to the alignment, in the DP table.
- CCAGTGC
CGATC
- [21-22 4a] [CSE 203] [Mark: 15]
131. At the beginning of the COVID-19 pandemic, two biologists Dr. Brown and Dr. William were trying to identify the similarity between COVID-19 and SARS viruses. To perform this task, they extracted the genome sequences of COVID-19 and SARS viruses and tagged them as S_1 and S_2 , respectively. Since they had a basic knowledge on algorithms, they planned to apply a dynamic programming based algorithm to guess an initial similarity between S_1 and S_2 . They found that finding the longest common subsequence of S_1 and S_2 or the edit distance between them can both be considered as good metrics of finding similarity between these two genome sequences. Therefore, they planned that Dr. Brown would find the longest common subsequence of S_1 and S_2 . In parallel, Dr. William would find the edit distance between S_1 and S_2 . Now as a student of algorithm course, your task is to carry out the role of Dr. Brown if your student ID is an odd number and the role of Dr. William if your student ID is an even number. To keep it simple, you have to consider only a substring of S_1 and S_2 , say $S'_1 = \langle \text{ATTGCAT} \rangle$ and $S'_2 = \langle \text{TAGCCT} \rangle$, respectively. You have to write the required recursive definition of your assigned problem and populate the corresponding dynamic programming table. Your table should contain the arrows to illustrate how the cell values are propagating. In case of longest common subsequence problem, you have to find the length of the longest common subsequence and the subsequence itself. On the other hand, in case of the edit distance problem, you have to show the edit distance cost and the sequence of required operations. As the set of available operations, you have to use the set {Insert (cost 1), Delete (cost 1), Replacement (cost 1), Keep (cost 0)} where the terms carry their traditional meanings. [20-21 7b] [CSE 203] [Mark: 15] [This question also contain Longest Common Substring topic]
132. Apply dynamic programming to calculate edit distance of the following two DNA sequences: $TGCATAT$ and $ATCCGA$, where, cost of insertion or deletion = 2, cost of substitution = 1. Write the recurrence relation and explain the intuition behind it, populate the DP table according to that recurrence relation and calculate the edit distance of the given DNA sequences from that table. [19-20 1a] [CSE 203] [Mark: 15]

0-1 Knapsack Problem

133. Solve the following instance of a 0-1 knapsack problem using Dynamic Programming.

Knapsack capacity = 6.

You have to show the dynamic programming table and find the optimal set of items using the dynamic programming table. [21-22 2b] [CSE 203] [Mark: 15]

Item	Weight	Value
1	2	8
2	1	7
3	2	12
4	2	6
5	2	10

134. Once an amateur thief entered into a museum full of tantalizing jewelry, geodes and rare gems. As he is new in this field, he brought just a single backpack with him that has a capacity of 6kg. The thief planned to get away with the most valuable objects without overloading the backpack. He found out that in a less-secured compartment, there were four (4) solid items, and their information are given in the following table.

Item	Weight	Value (USD)
Brooch of the Queen	3	15
Crown of the King	2	20
Decorated vase	10	30
An ancient coin	2	14

Since the items are unbreakable, for each item, the thief must pick the whole of it or not consider it at all. Now although our thief is amateur in this business, he possesses some algorithmic knowledge. Therefore, he applied a greedy technique and selected the items that gave him the maximum possible profit per weight, and then he left the museum with so much zeal! Now, using the information of the backpack of the thief and the item table given for this question, your first task is to show the steps of finding the maximum possible profit and the corresponding list of items using the technique that the thief adopted. You have also identified that the scenario at the museum was a special one and only dynamic programming is able to find the optimal solution in that case. Therefore, your second task is to find the optimal profit and the list of items using dynamic programming. [20-21 5a] [CSE 203] [Mark: 20]

135. Consider the following instance of the 0/1 knapsack problem. Solve this instance using dynamic programming.

Assume that the knapsack capacity is 7. You have to show the dynamic programming table and find all solutions using this table. [18-19 1c] [CSE 203] [Mark: 15]

Item	Weight	Value
1	1	50
2	3	60
3	5	100
4	2	70
5	2	80

Activity Selection Problem

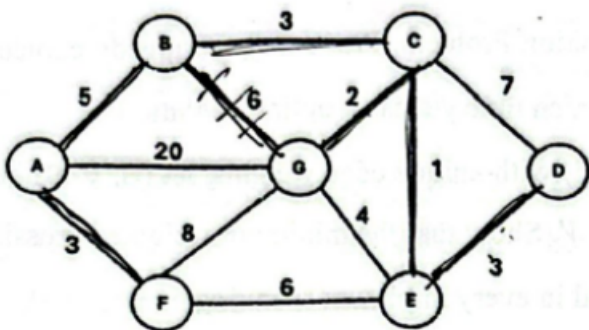
136. Define the Activity Selection Problem. Prove that the greedy choice of selecting the activities based on earliest finish time yields an optimal solution. [23-24 7a] [Mark: 15]
137. Consider a set of n intervals, with each interval i specifying a start time s_i and a finish time $f_i (s_i < f_i)$. Two intervals are compatible if they do not overlap. Give a greedy algorithm to select a compatible subset of intervals of maximum possible size. Prove the correctness, and analyze the running time of your algorithm. [21-22 1a] [CSE 203] [Mark: 16]
138. Suppose that we have a set of n activities, $A = \{a_1, a_2, \dots, a_n\}$, where each activity a_i has a start time s_i and a finish time $f_i (0 \leq s_i \leq f_i < \infty)$. Two activities are compatible if and only if their intervals do not overlap. Our target is to generate the maximum-sized subset of mutually compatible activities. Now, prove that an optimal solution is always guaranteed if we design a greedy algorithm by considering the activity with the earliest finish time from the remaining activities. [20-21 7a] [CSE 203] [Mark: 10]
139. Suppose that you run a bus lending business, and each request is at the format: (start time, end time). A bus can be used by one user at a time. Suppose that you have got the following bus-one requests for the next day.
 $\{[1, 6), [4, 8), [6, 8), [9, 15), [6, 9), [11, 15), [2, 6)\}$.
 Find all solutions of the maximum number of requests that can be satisfied by using
- earliest start time
 - shortest interval, and
 - earliest end time.
- [18-19 2b] [CSE 203] [Mark: 12]

Fractional Knapsack Problem

140. Show that the fractional knapsack problem has the optimal substructure property. Also show that the 0-1 knapsack problem has the overlapping subproblems property. [18-19 3b] [CSE 203] [Mark: 10]

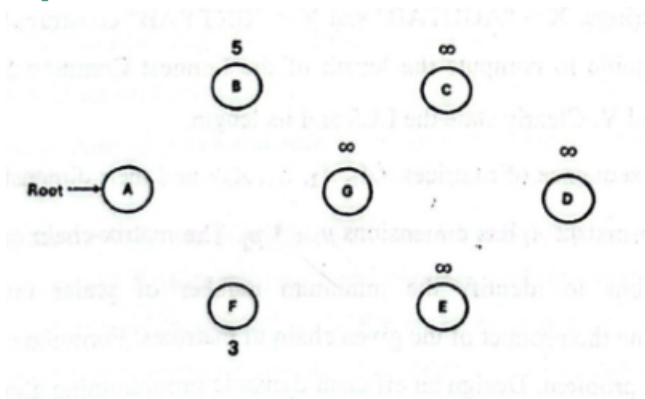
Minimum Spanning Tree

141. Apply Prim's algorithm to find the minimum spanning tree of the following graph.

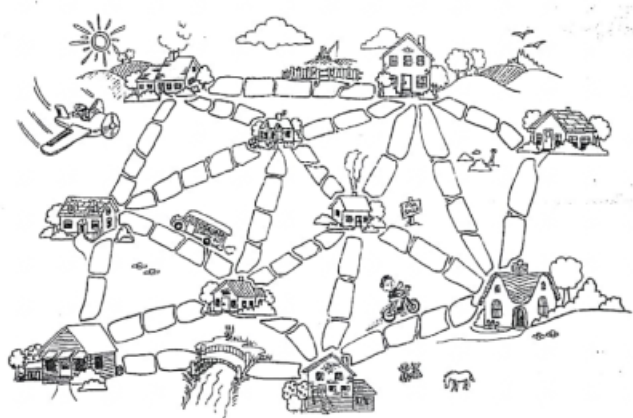


Start with node A as the root node. At each step, show the intermediate MST (i.e., the partial tree) and show the estimated cost to connect each non-visited node (i.e., the minimum weight of an edge connecting an unvisited node to any visited nodes). The first step, where only the root node is included in the tree, is done for you. [23-24 5c] [Mark: 10]

10]



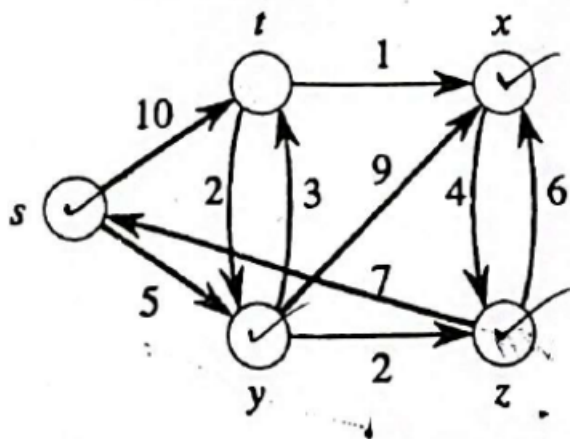
142. Given a graph, $G = (V, E)$ with unique edge weights, let $(S, V-S)$ be a cut in G such that S is non-empty and $S \neq V$. Show that the minimum weighted crossing edge between any such cut in G , is included in every minimum spanning tree. [23-24 7b] [Mark: 10]
143. Argue in favor of the correctness of the Prim's algorithm to find a minimum spanning tree of a graph $G = (V, E)$. Prove any non-trivial graph properties used in the argument. [22-23 7a] [Mark: 15]
144. Once upon a time there was a city that had no proper roads. Getting around the city was particularly difficult after rainstorms because the ground used to become very muddy, cars got stuck in the mud, and people got their boots dirty. The mayor of the city decided that some of the streets must be paved, but he did not want to spend more money than necessary because the city also wanted to build a swimming pool. The mayor therefore specified two conditions:
- Enough streets must be paved so that it is possible for everyone to travel from their house to anyone else's house only along the paved roads, and
 - The paving should cost as little as possible.
- The following figure shows the layout of the city. The number of paving stones in between two houses represents the cost of paving that route. Your task is to apply a greedy based algorithm that you have learned under this course to fulfill the requirement of the Mayor. Note that the bridge in the figure should not be considered as a paving stone. In your answer script, you must draw a simple graph representing the given layout of the muddy city. After that, you have to show the steps of applying your chosen greedy algorithm to find the connected subgraph which contains all the houses of the city (considering them as vertices of your drawn graph) and a subset of roads (considering them as edges) in a way that the total number of paving stones required is the minimum. You have to ensure that your connected subgraph does not contain any cycle, and you need to mention the algorithm that you are adopting to find your desired solution. [20-21 5b] [CSE 203] [Mark: 15]



Greedy Algorithms

145. Find the shortest path distance from node x to node t in the following graph using the Dijkstra's Algorithm. Re-draw the diagram for each intermediate step of the algorithm (after relaxation) while annotating each node with appropriate

shortest path distance estimate of that step. [22-23 7b] [Mark: 10]



146. Given a set $U = \{x_1, x_2, \dots, x_n\}$ of n integers and an integer k ($k < n$), give an optimal greedy algorithm to find a subset S of size k in U such that the summation of the numbers in S is maximum (over all the subsets of size k in U). Prove the correctness of your algorithm, and analyze the running time. [21-22 7a] [Mark: 15]
147. Given a set $S = \{x_1, x_2, \dots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \dots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contains all the numbers in S . For example, an interval $[1.5, 2.5]$ is a unit-length interval (since $2.5 - 1.5 = 1$), and it contains the real numbers from 1.5 to 2.5. You have to find the smallest set of such unit length intervals which contains all the real numbers in S . Prove the correctness of your algorithm, and analyze the running time. [21-22 3b] [CSE 203] [Mark: 15]
148. A set of tasks with deadlines and the total profit earned on completing each task have been given below. Assume that a task takes one unit of time to execute, and it can't execute beyond its deadline. Also assume that only a single task will be executed at a time. Write an algorithm to calculate the maximum profit earned by executing the tasks within the specified deadlines and calculate the maximum profit from the given data using the algorithm. [19-20 1b] [CSE 203] [Mark: 20]

Task	Deadline	Profit
T1	7	9
T2	3	14
T3	2	5
T4	6	10
T5	8	3
T6	5	15
T7	6	5
T8	3	7
T9	2	8
T10	4	6

149. Dream coins are used by the people of the Dreamland. Assume that only the following coins are available at this land: \$1, \$7, \$11, and \$15.
- Write an example of an amount where the greedy strategy for the Coin-Change problem does not provide an optimal solution. You must mention the optimal solution, and the greedy solution.
 - By using Dynamic Programming, find the minimum number of coins that add up to a given amount of money of \$20. [18-19 1a] [CSE 203] [Mark: 10]

Disjoint Set & Union-Find

150. Consider the array implementation of the Union-Find data structure for some set S of size n , where union keep the name of the largest set. Analyze the run-time complexity of any sequence of k Union operations. [23-24 8c] [Mark: 10]
151. Consider the array implementation of the Union-Find data structure for some set S of size n , where unions keep the name of the larger set. Analyze the run-time complexity of any sequence of k Union operations. [22-23 7c] [Mark: 10]
152. Write an algorithm to detect cycle in an undirected graph using disjoint set data structure. Mention the cases where this algorithm will perform more efficiently than BFS or DFS based approach. [19-20 4a] [CSE 203] [Mark: 15]
153. Explain, with examples, the union-by-rank and the path-compression heuristics that are used in UNION and FIND operations of disjoint-set data structures. [18-19 4a] [CSE 203] [Mark: 10]

Miscellaneous

154. You are given access to the following functions of a list data structure:
- `init(n)` - Initialize an array $L[0..n-1]$ of length n for integers. Current position 'points to' $L[0]$. All the functions below are applied on this array which is not accessible directly.
 - `insert(item)` - Inserts an element at the current position and current position is 'incremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). Here, `item` is the element to be inserted. In case the list is full, it returns ERR (a large negative constant).
 - `remove()` - Removes the element at the current position and current position is 'decremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). In case the list is empty, it returns ERR (a large negative constant).
 - `value()` - Returns the value of the element at the current position. Current position remains unchanged. In case the list is empty, it returns ERR (a large negative constant).
 - `moveToStart()` - Sets the current position to $L[0]$
 - `moveToEnd()` - Sets the current position to $L[n-1]$.
 - `next()` - Increments the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). If $k = n-1$ it returns ERR (a large negative constant).
 - `prev()` - Decrements the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). If $k = 0$, it returns ERR (a large negative constant).
- Now you have to build one queue and one stack in one List data structure as defined above. The queue should grow from the beginning of the list (i.e., first element of the queue should be at $L[0]$) and the stack should grow from the end of the list (i.e., the first element of the stack should be at $L[n-1]$). You have no access to the code of the above implementation and can only call these functions in your code. You need to write appropriate codes (no specific programming language is required; pseudo code is fine) for implementing the queue and stack simultaneously as described above. Please explain your code by giving appropriate comments. In particular, you need to implement the following four functions:
- `push(item)` : push a positive integer, `item` in the stack. If there is an error (e.g., no more space available for growing the stack in the list, it should output an appropriate error message and returns with -1.
 - `pop()` : pop an element from the stack. If there is an error, it should output an appropriate error message and returns with -1.
 - `enqueue(item)` : enqueue a positive integer, `item` in the queue. If there is an error (e.g., no more space available for growing the queue in the list, it should output an appropriate error message and returns with -1.
 - `dequeue()` : dequeue an element from the queue. If there is an error, it should output an appropriate error message and returns with -1. [21-22 3c] [Mark: 25]
155. You are given a tree of size n as an array `parent[0..n-1]` where every index i in the `parent[]` represents a node and the value at i represents the immediate parent of that node. For root node the value will be -1. Write an algorithm to find the height of the generic tree given the parent array.

Input: parent[] = {-1, 0, 0, 0, 3, 1, 1, 2}

Output: 2

[21-22 6a] [CSE 203] [Mark: 15]

156. Given a linked list where in addition to the next pointer, each node has a child pointer, which may or may not point to a separate list. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure, as shown in Figure for Q. 8(a). You are given the head of the first level of the list. Flatten the list so that all the nodes appear in a single-level linked list. You need to flatten the list in a way that all nodes at the first level should come first, then nodes of second level, and so on.

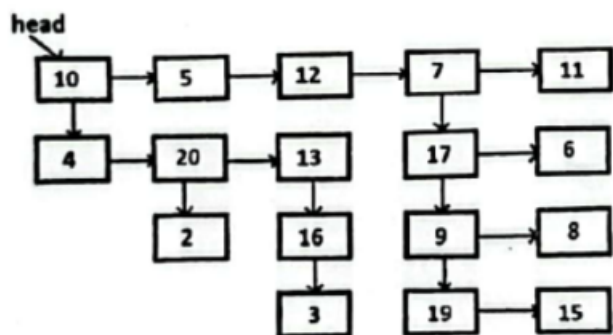


Figure for Q. 8(a)

The above list should be converted to $10 \rightarrow 5 \rightarrow 12 \rightarrow 7 \rightarrow 11 \rightarrow 4 \rightarrow 20 \rightarrow 13 \rightarrow 17 \rightarrow 6 \rightarrow 2 \rightarrow 16 \rightarrow 9 \rightarrow 8 \rightarrow 3 \rightarrow 19 \rightarrow 15$. [21-22 8a] [CSE 203] [Mark: 15]

157. $G = (V, E)$ is a connected graph with the vertex set V and edge set E , and G is represented using adjacency-list. A spanning tree of the graph G is a tree that spans G , that means, it includes every vertex of G and is a subgraph of G (every edge in the tree belongs to E). The cost of the spanning tree is the sum of the weights of all the edges in the tree. There can be many spanning trees of the same graph. Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees. There also can be many minimum spanning trees of the same graph. Now, in the following figure, you have been given with the Prim's algorithm to find the minimum spanning tree of any graph G . It uses the traditional Binary Min-Heap to maintain a minimum priority queue. Your task is to analyze the runtime complexity of this given algorithm. In your analysis, you have to mention the individual runtime of each of the significant operations. How this running time is improved if we decide to use a Fibonacci Heap instead?

```

MST-PRIM( $G, w, r$ )
1 for each  $u$  in  $G.V$ 
2   ... $u.key = \text{infinity}$ 
3   ... $u.pi = \text{NIL}$ 
4  $r.key = 0$ 
5  $Q = G.V$ 
6 while  $Q \neq \text{empty}$ 
7   ... $u = \text{EXTRACT-MIN}(Q)$ 
8   ...for each  $v$  in  $G.Adj[u]$ 
9     ...if  $v$  in  $Q$  and  $w(u,v) < v.key$ 
10    ... $v.pi = u$ 
11    ... $v.key = w(u,v)$ 
//the pseudo-code is slightly changed
  
```

[20-21 7c] [CSE 203] [Mark: 10]

158. Given two arrays $a[]$ and $b[]$, each containing N distinct points in the plane, design an algorithm whose running

time is linear under reasonable assumptions and uses at most linear extra space to determine whether the two arrays contains precisely the same set of points (but possibly in a different order). [18-19 5d] [CSE 203] [Mark: 8]